BIT MANIPULATION

Foundation & Templates

>

💡 WHY BIT MANIPULATION?

Bitwise operations manipulate individual bits directly inside CPU registers in **O(1) time with 0 extra memory**! They power low-level systems, cryptographic algorithms, and optimal interview solutions.

💡 THE KERNIGHAN'S TRICK AHA

`n & (n - 1)` clears the lowest set bit (`1`) of `n` in exact O(1) time! Use this to count set bits in O(\text{set bits}) instead of 32 loop iterations!

1 6 ESSENTIAL BIT OPERATORS

- **AND (`&`):** Both bits 1 \rightarrow 1. Masking.
- **OR (`|`):** Either bit 1 \rightarrow 1. Setting bits.
- **XOR (`^`):** Different bits \rightarrow 1. $x \oplus x = 0, x \oplus 0 = x$!
- **NOT (`~`):** Bitwise inversion.
- **Left Shift (`<<`):** Multiply by 2^k.
- **Unsigned Right Shift (`>>>`):** Fills 0 on left.

2 VISUALIZING KERNIGHAN'S TRICK

Clearing lowest set bit of 12 (`1100`):

```
n     = 12 -> 1100 (binary)
n - 1 = 11 -> 1011 (binary)
n & (n - 1) = 1000 (binary) -> 8!
Lowest set bit at position 2 is CLEAR
ED!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation / Problem	Time Complexity	Auxiliary Space
Single Number (XOR Accumulation)	**O(N) single pass**	**O(1) space**
Hamming Weight (n & (n-1))	**O(set bits) <= 32**	**O(1) space**
Counting Bits (DP + Bitshift)	**O(N) single pass**	**O(N) result array**
Sum of 2 Integers (XOR + AND Carry)	**O(32) = O(1)**	**O(1) space**

⚡ CLUES THAT SCREAM BIT MANIPULATION

1. *"Every element appears twice except one"* \rightarrow Accumulative XOR (`a ^= num`)
2. *"Count total 1 bits in 32-bit integer"* \rightarrow Kernighan's `n = n & (n - 1)`
3. *"Add two numbers without + or -"* \rightarrow Bitwise XOR (`a ^ b`) + Carry (`(a & b) << 1`)

>

1 5 MUST-KNOW BIT TRICKS

```
// 1. Get k-th bit (0 or 1):
int bit = (n >> k) & 1;

// 2. Set k-th bit to 1:
n = n | (1 << k);

// 3. Clear k-th bit to 0:
n = n & ~(1 << k);

// 4. Toggle k-th bit:
n = n ^ (1 << k);

// 5. Check if n is power of 2:
boolean isPowerOfTwo = (n > 0) && ((n & (n - 1)) == 0);
```

2 KERNIGHAN'S BIT COUNTING HELPER

```
public static int hammingWeight(int n) {
  int count = 0;
  while (n != 0) {
    n = n & (n - 1); // Clears lowest set bit!
    count++;
  }
  return count;
}
```

Loops ONLY for set bits (`1`), skipping all 0 bits!

BIT MANIPULATION

Core Patterns — Part 1 & Part 2

>

PATTERN 1 SINGLE NUMBER (XOR Self-Cancellation) e.g. Single Number (LC 136)

XOR CANCELLATION MATH

$x \oplus x = 0 and x \oplus 0 = x$. Since all duplicate pairs cancel to 0, XORing all array numbers leaves ONLY the unique single number!

TEMPLATE — LC 136

```
public int singleNumber(int[] nums) {
  int res = 0;
  for (int num : nums) {
    res ^= num; // Pair cancellation!
  }
  return res;
}
```

PATTERN 2 MISSING NUMBER (Index XOR Array Accumulation) e.g. Missing Number (LC 268)

INDEX MATCHING STRATEGY

XOR all indices 0..N with all array values `nums[i]`. Every present number cancels its matching index, leaving ONLY the missing number!

TEMPLATE — LC 268

```
public int missingNumber(int[] nums) {
  int res = nums.length;
  for (int i = 0; i < nums.length; i++) {
    res ^= i ^ nums[i];
  }
  return res;
}
```

>

PATTERN 3 COUNTING BITS (DP Bitshift Recurrence) e.g. Counting Bits (LC 338)

RECURRENCE FORMULA

`dp[i] = dp[i >> 1] + (i & 1)`. `i >> 1` shifts off the last bit, which was computed earlier! O(N) time.

TEMPLATE — LC 338

```
public int[] countBits(int n) {
  int[] res = new int[n + 1];
  for (int i = 1; i <= n; i++) {
    res[i] = res[i >> 1] + (i & 1);
  }
  return res;
}
```

PATTERN 4 REVERSE BITS (32-Bit Reversal Accumulator) e.g. Reverse Bits (LC 190)

BIT SHIFT ACCUMULATOR

Loop 32 times: shift `res` left by 1, OR with lowest bit `n & 1`, shift `n` right by 1 (`n >>>= 1`).

TEMPLATE — LC 190

```
public int reverseBits(int n) {
  int res = 0;
  for (int i = 0; i < 32; i++) {
    res = (res << 1) | (n & 1);
    n >>>= 1; // Unsigned right shift!
  }
  return res;
}
```

BIT MANIPULATION

Core Patterns — Part 3 & Part 4

>

PATTERN 5 SUM OF TWO INTEGERS (XOR Sum + AND Carry) e.g. Sum of Two Integers (LC 371)

HALF-ADDER HARDWARE LOGIC

`a ^ b` computes addition WITHOUT carry.
`(a & b) << 1` computes carry.
Repeat until carry `b == 0`!

TEMPLATE — LC 371

```
public int getSum(int a, int b) {
  while (b != 0) {
    int carry = (a & b) << 1;
    a = a ^ b;
    b = carry;
  }
  return a;
}
```

>

PATTERN 6 REVERSE INTEGER (32-Bit Overflow Guard) e.g. Reverse Integer (LC 7)

OVERFLOW GUARD FORMULA

Check `res > Integer.MAX_VALUE / 10` or `res < Integer.MIN_VALUE / 10` before multiplying by 10!

TEMPLATE — LC 7

```
public int reverse(int x) {
  int res = 0;
  while (x != 0) {
    int pop = x % 10; x /= 10;
    if (res > Integer.MAX_VALUE / 10 || (res == Integer.MAX_VALUE / 10 && pop >
7)) return 0;
    if (res < Integer.MIN_VALUE / 10 || (res == Integer.MIN_VALUE / 10 && pop < -
8)) return 0;
    res = res * 10 + pop;
  }
  return res;
}
```

BIT MANIPULATION

Decision Tree & Trigger Words

>

BIT MANIPULATION — WHICH TRICK TO USE?

Start: What is the bitwise goal?

🎯 **Find single non-duplicate number**	→	**Accumulative XOR**	`res ^= num` (pairs cancel
🔢 **Count total 1 bits (Hamming weight)**	→	**Kernighan's Trick**	`n = n & (n - 1)`
📊 **Count bits for all 0..N numbers**	→	**DP Bitshift**	`dp[i] = dp[i >> 1] + (i & 1)`
➕ **Add numbers without + or -**	→	**XOR + AND Carry**	`a ^ b` and `(a & b) << 1`

🎯 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Single number / unique element"	XOR Accumulation	`res ^= num`
"Number of 1 bits"	Kernighan's	`n = n & (n - 1)`
"Count set bits for 0..N"	DP Bitshift	`res[i] = res[i >> 1] + (i & 1)`
"Sum without + operator"	Bitwise Addition	Loop `a = a ^ b`, `b = carry`

💼 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why `>>>` instead of `>>` in Reverse Bits?	`>>>` is unsigned right-shift (fills 0 on left). `>>` preserves sign bit (fills 1 for negative numbers)!
How to check if N is power of 2?	`(n > 0) && ((n & (n - 1)) == 0)`. Powers of 2 have exactly ONE set bit!

BIT MANIPULATION

FAANG Problem Ladder (All 10 Problems)

>

EASY — XOR & Bit Counting

LC 136 · Single Number — EASY

Pattern: XOR Self-Cancel
Key Insight: `res ^= num`.

```
public int singleNumber(int[] nums) {
  int res = 0;
  for (int n : nums) res ^= n;
  return res;
}
```

LC 191 · Number of 1 Bits — EASY

Pattern: Kernighan's Trick
Key Insight: `n = n & (n - 1)`.

```
public int hammingWeight(int n) {
  int c = 0;
  while (n != 0) { n &= (n - 1); c++; }
  return c;
}
```

LC 268 · Missing Number — EASY

Pattern: Index XOR Match
Key Insight: `res ^= i ^ nums[i]`.

```
public int missingNumber(int[] nums) {
  int res = nums.length;
  for (int i = 0; i < nums.length; i++) res ^= i ^ nums[i];
  return res;
}
```

EASY & MEDIUM — Counting Bits & Reverse

LC 338 · Counting Bits — EASY

Pattern: DP Bitshift
Key Insight: `dp[i] = dp[i >> 1] + (i & 1)`.

```
public int[] countBits(int n) {
  int[] res = new int[n + 1];
  for (int i = 1; i <= n; i++) res[i] = res[i >> 1] + (i & 1);
  return res;
}
```

LC 190 · Reverse Bits — EASY

Pattern: 32-Bit Unsigned Shift
Key Insight: `res = (res << 1) | (n & 1)`, `n >>>= 1`.

```
public int reverseBits(int n) {
  int res = 0;
  for (int i = 0; i < 32; i++) {
    res = (res << 1) | (n & 1); n >>>= 1;
  }
  return res;
}
```

>

MEDIUM — Bitwise Addition & Overflow Reverse

LC 371 · Sum of Two Integers — MEDIUM

Pattern: XOR + AND Carry
Key Insight: `a = a ^ b`, `b = (a & b) << 1`.

```
public int getSum(int a, int b) {
  while (b != 0) {
    int carry = (a & b) << 1;
    a = a ^ b; b = carry;
  }
  return a;
}
```

MEDIUM — Integer Reversal

LC 7 · Reverse Integer — MEDIUM

Pattern: Overflow Protection
Key Insight: Guard `res > MAX / 10` before multiplying.

```
public int reverse(int x) {
  int res = 0;
  while (x != 0) {
    int pop = x % 10; x /= 10;
    if (res > Integer.MAX_VALUE / 10 || res < Integer.MIN_VALUE / 10) return 0;
    res = res * 10 + pop;
  }
  return res;
}
```

COMPLETE BIT MANIPULATION PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
136	Single Number	XOR Self-Cancellation	O(N)	O(1)	E
191	Number of 1 Bits	Kernighan's Trick	O(set bits)	O(1)	E
268	Missing Number	Index XOR Match	O(N)	O(1)	E
338	Counting Bits	DP Bitshift Recurrence	O(N)	O(N)	E
190	Reverse Bits	32-Bit Unsigned Shift	O(32)	O(1)	E

BIT MANIPULATION

Dry Run, Proofs & Cheat Sheet

>

🔍 DRY RUN — Single Number ([4, 1, 2, 1, 2])

Step	num	XOR Operation (`res ^= num`)	res Value
1	4	`0 ^ 4`	4
2	1	`4 ^ 1`	5
3	2	`5 ^ 2`	7
4	1	`7 ^ 1` (cancels previous 1)	6
5	2	`6 ^ 2` (cancels previous 2)	4 ✅

📐 MATHEMATICAL PROOF — KERNIGHAN'S CLEAR BIT FORMULA

Claim: `n & (n - 1)` clears the lowest set bit of `n`.

Proof: Subtracting 1 flips the rightmost `1` to `0` and all lower trailing `0`s to `1`s. Performing `n & (n - 1)` keeps all higher bits unchanged while forcing the rightmost `1` to `0` \rightarrow **Exact 1 bit cleared!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Code Single Number in 1m with XOR
- ☐ Write Kernighan's `n & (n - 1)` bit counter
- ☐ Code Missing Number with index XOR
- ☐ Write Counting Bits with DP bitshift
- ☐ Code Reverse Bits using `>>>` shift
- ☐ Write Sum of Two Integers (XOR + Carry)
- ☐ Write Reverse Integer with overflow guard
- ☐ Prove why `n & (n - 1) == 0` tests power of 2

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Operator Precedence Warning

Bitwise operators (`&`, `|`, `^`) have LOWER precedence than comparison operators (`==`, `<`)! ALWAYS wrap bitwise expressions in parentheses: `((n & 1) == 0)`!

Rule 2 — Always Use Unsigned Right Shift for Reversal

In 32-bit bit manipulation, use `>>>` instead of `>>` so negative numbers do not fill leading 1s!